

parameter passing methods

- In C, a function specifies the modes of parameter passing to it. There are two ways to specify function calls: call by value and call by reference in C.
- In call by value, the function parameters gets the copy of actual parameters which means changes made in function parameters did not reflect in actual parameters.
- In call by reference, the function parameter gets reference of actual parameter which means they point to similar storage space and changes made in function parameters will reflect in actual parameters.

Difference Between Argument and Parameter in C

Argument	Parameter
Also known as actual parameters	Also known as formal parameters
Arguments are used while calling the function	Parameters are used during the declaration of the function
Argument is the actual value of this variable that gets passed to function.	Parameter is variable in the declaration of the function.

difference between argument and parameter is that parameter is a variable defined during function declaration while arguments are the actual values.

Difference Between Call by Value and Call by Reference in C

Calling by Value	Calling by Reference
Copies the value of an object.	Pass a pointer that contains the memory address of an object that gives access to its contents.
Guarantees that changes that alter the state of the parameter will only affect the named parameter bounded by the scope of the function.	Changes that alter the state of the parameter will reflect to the contents of the passed object.
Simpler to implement and simpler to reason with.	More difficult to keep track of changing values that happens for each time a function may be called.

Call by Value in C

Calling a function by value will cause the program to copy the contents of an object passed into a function. To implement this in C, a function declaration has the following form: [return type] functionName([type][parameter name],...).

```
#include <stdio.h>
```

```
void swap(int x, int y){  
    int temp = x;  
    x = y;  
    y = temp;  
}
```

```
int main(){  
    int x = 10;  
    int y = 11;  
    printf("Values before swap: x = %d, y = %d\n", x,y);  
    swap(x,y);  
    printf("Values after swap: x = %d, y = %d", x,y);  
}
```

- **Values before swap: x = 10, y = 11**
- **Values after swap: x = 10, y = 11**

observe that even when we change the content of x and y in the scope of the swap function, these changes do not reflect on x and y variables defined in the scope of main. This is because we call swap() by value and it will get separate memory for x and y so the changes made in swap() will not reflect in main().

Call by Reference in C

Calling a function by reference will give function parameter the address of original parameter due to which they will point to same memory location and any changes made in the function parameter will also reflect in original parameters. To implement this in C, a function declaration has the following form: [return type] functionName([type]* [parameter name],...).

```
#include <stdio.h>
```

```
void swap(int *x, int *y){  
    int temp = *x;  
    *x = *y;  
    *y = temp;  
}
```

Values before swap: x = 10, y = 11
Values after swap: x = 11, y = 10

```
int main(){  
    int x = 10;  
    int y = 11;  
    printf("Values before swap: x = %d, y = %d\n", x,y);  
    swap(&x,&y);  
    printf("Values after swap: x = %d, y = %d", x,y);  
}
```

observe in function parameters instead of using `int x, int y` we used `int *x, int *y` and in function call instead of giving `x, y`, we give `&x, &y` this methodology is called by reference as we used pointers as function parameter which will get original parameters' address instead of their value. `&` operator is used to give address of the variables and `*` is used to access the memory location that pointer is pointing. As the function variable is pointing to the same memory location as the original parameters, the changes made in `swap()` reflect in `main()` which we can see in the above output.

When to Use Call by Value and Call by Reference in C?

- Copying is expensive, and we have to use our resources wisely. Imagine copying a large object like an array with over a million elements only to enumerate the values inside the array, doing so will result in a waste of time and memory. Time is valuable and we can omit to copy when:
- We intend to read state information about an object, or
- Allow a function to modify the state of our object.
- However, when we do not intend our function to alter the state of our object outside of our function, copying prevents us from making unintentional mistakes and introduce bugs.

Advantages of Using Call by Value Method

- Guarantees that changes that alter the behavior of a parameter stay within its scope and do not affect the value of an object passed into the function
- Reduce the chance of introducing subtle bugs which can be difficult to monitor.
- Passing by value removes the possible side effects of a function which makes your program easier to maintain and reason with.

Advantages of Using Call by Reference Method

Calling a function by reference does not incur performance penalties that copying would require. Likewise, it does not duplicate the memory necessary to access the content of an object that resides in our program.

Allows a function to update the value of an object that is passed into it.

Allows you to pass functions as references through a technique called function pointers which may alter the behavior of a function. Likewise, lambda expressions may also be passed inside a function. Both enable function composition which has neat theoretical properties.

Disadvantages of Using Call by Value Method

- Incurs performance penalty when copying large objects.
- Requires to reallocate memory with the same size as the object passed into the function.

Disadvantages of Using Call by Reference Method

- For every function that shares with the same object, your responsibility of tracking each change also expands.
- Making sure that the object does not die out abruptly is a serious issue about calling a function by reference. This is especially true in the context of a multithreaded application.

